



Fundamentos de Compiladores

Sistemas de Informação – UNEB
Prof. Antonio Atta

Sistema de Execução
Máquina de Pilha - MP

Conceitos iniciais

- ❑ O fase de **síntese** de um compilador consiste na geração do código objeto a partir do programa fonte devidamente analisado
- ❑ O código objeto normalmente não corresponde à linguagem de máquina de um computador real – a tradução seria uma tarefa muito trabalhosa
- ❑ Solução:



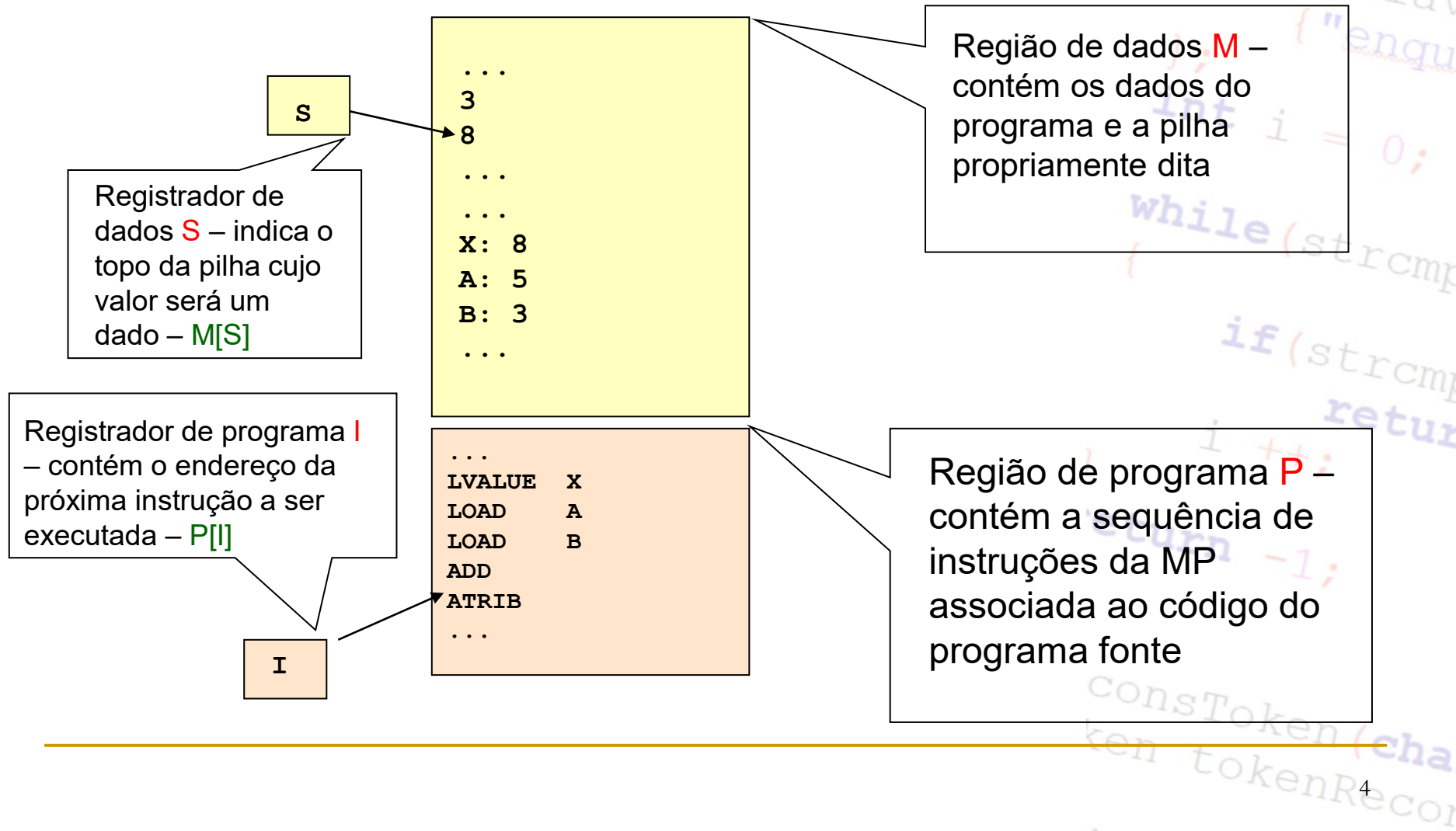
Criar uma máquina hipotética, mais adequada para as finalidades de tradução, conhecida como **sistema de execução** (**run-time system**)

Conceitos iniciais

- ❑ As instruções da máquina hipotética, correspondentes ao programa em processo de tradução, são denominadas **código intermediário** e podem ser traduzidas, posteriormente, para o **código executável** na máquina alvo
- ❑ Um sistema de execução muito utilizado para linguagens modernas, que dão suporte a construções semanticamente complexas, como modularização de código em procedimentos e recursividade, é conhecida como **Máquina de Pilha – MP**

Máquina de Pilha – MP

■ Características gerais de uma MP possível



Máquina de Pilha – MP

■ Algoritmo de execução da MP

- ❑ Carregar o programa na região **P**
- ❑ Inicializar os registradores **I** e **S**
- ❑ Para cada instrução do programa
 - Executar a instrução (interromper o programa no caso de instrução de parada)
 - Incrementar registrador **I** (exceto nas instruções de desvio)

Instruções da MP

■ Instruções manipulação de dados

PUSH v	- Empilha a constante v no topo da pilha
POP	- Desempilha o valor no topo da pilha
COPY	- Duplica o valor do topo da pilha
STOR x	- Desempilha e armazena o conteúdo do topo da pilha na área de dados x
LOAD y	- Empilha o valor da área de dados y

■ Instruções aritméticas inteiras

ADD	- Desempilha topo e sub-topo, soma e empilha resultado
SUB	- Desempilha topo e sub-topo, subtrai (sub-topo – topo) e empilha resultado
MUL	- Desempilha topo e sub-topo, multiplica e empilha resultado
DIV	- Desempilha topo e sub-topo, divide (sub-topo / topo) e empilha resultado

Instruções da MP

■ Instruções aritméticas de ponto flutuante

ADDF	- Desempilha topo e sub-topo, soma e empilha o resultado
SUBF	- Desempilha topo e sub-topo, subtrai (sub-topo – topo) e empilha o resultado
MULF	- Desempilha topo e sub-topo, multiplica e empilha o resultado
DIVF	- Desempilha topo e sub-topo, divide (sub-topo / topo) e empilha o resultado

■ Instruções de controle de fluxo

LABEL L	- Rotula a próxima instrução com rótulo L
GOTO L	- Desvia para a instrução rotulada com rótulo L
GOFALSE L	- Desempilha e desvia o fluxo de execução para a instrução rotulada L se o valor for igual a zero
GOTRUE L	- Desempilha e desvia o fluxo de execução para a instrução rotulada L se o valor for maior que zero

Instruções da MP

■ Instrução de fim de programa

HALT - Para a execução do programa

■ Instruções de E/S

GET_I - Lê um valor inteiro da entrada padrão e coloca no topo da pilha

GET_F - Lê um valor real da entrada padrão e coloca no topo da pilha

GET_C - Lê um caracter da entrada padrão e coloca no topo da pilha

PUT_I - Desempilha valor inteiro e escreve na saída padrão

PUT_F - Desempilha valor real e escreve na saída padrão

PUT_C - Desempilha um caracter e escreve na saída padrão

Instruções da MP

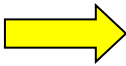
- Instrução de início de programa

INIP – Início de programa principal ($S = -1$)

- Instrução de alocação de memória

AMEM m – Aloca m posições de memória ($S = S + m$)

Exemplo:

`int i,j,k;` }  **AMEM 3**

Uso da MP

- Suporte da MP a programas sem blocos (apenas o bloco *main*)
 - Com o conjunto de instruções visto, a MP dá pleno suporte à geração de código e execução de programas sem blocos além do *main*
- Suporte da MP a programas com blocos sem parâmetros e não recursivos
 - Instruções de chamada e retorno de blocos

CALL L – Desvia para o procedimento identificado pelo rótulo L
(**S** = **S**+1; **M**[**S**]=**I**+1; **I**=**L**)

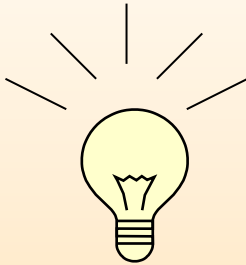
RET – Retorna do procedimento atual para o procedimento chamador
(**I** = **M**[**S**]; **S** = **S** – 1)

Uso da MP

- Suporte da MP a programas com blocos recursivos sem parâmetros
 - Requer suporte da MP a várias “instâncias” de execução do bloco
 - Não há previsão do número de instâncias – elemento definido apenas em **tempo de execução**. Como resolver?
 - Propriedades que indicam a solução:
 - A cada instância de recursão só se tem acesso à última alocação de variáveis locais do bloco
 - O processo de criação e destruição das diversas alocações de variáveis locais de um bloco segue uma disciplina de pilha

Uso da MP

❑ Solução:



Usar a própria pilha !!!

Uso da MP

❑ Implementação: Endereço Relativo

- As posições alocadas para os diversos tipos de variáveis (globais, locais, parâmetros de blocos) são relativas a um endereço base
- O acesso ao conteúdo dessas variáveis ocorre através das componentes endereço base e deslocamento (positivo ou negativo)
- Um conjunto de registradores base mantém os endereços relativos associadas a cada instância de execução do bloco

Uso da MP

■ Implementação: Endereço Relativo (cont.)

- Existe um registrador base para cada nível (escopo) de execução de um bloco que é compartilhado por todas as instâncias de um mesmo nível
- Em chamadas recursivas, por exemplo, a instância em execução obtém o direito de usar o registrador base do seu nível (escopo)
- A cada chamada recursiva ou de bloco do mesmo nível (escopo) do bloco atual, o valor atual do registrador base, de uso comum, é salvo na pilha e restaurado no retorno

Uso da MP

❑ Implementação: Endereço Relativo (cont.)

- No caso da implementação da MP para o projeto Do::Block, apenas dois registradores de base serão necessários – um para as variáveis globais (nível 0) e outro para as instâncias de execução dos blocos (inclusive o *principal*)
- Em implementações da MP para linguagens como Pascal, que possuem mais de dois níveis de execução de procedimentos, um número maior de registradores de base é necessário
- A região na pilha apontada por um registrador base é conhecida como registro de ativação do bloco

Uso da MP

❑ Vantagem do Endereço Relativo

- Dá suporte simples à passagem de parâmetros por valor e por referência
- *Overhead* reduzido na chamada e retorno de blocos
- Modelo adotado na maioria dos processadores para implementar passagem de parâmetros (construção de alto nível) em linguagem de máquina

❑ Desvantagens do Endereço Relativo

- Necessidade da estrutura de registros de base

Uso da MP

❑ Estrutura de Registradores de Base

- Usamos um vetor extra D onde cada posição corresponde ao registrador de base de um nível (escopo) do programa
- O nível do procedimento indica o índice a ser usado para acessar o conteúdo do seu registrador base



Uso da MP

- ❑ Comandos da MP para suporte ao Endereçamento Relativo
 - Alocação e liberação de memória para variáveis

AMEM m – Como antes. Serve tanto para alocação de memória para variáveis globais (nível 0) quanto para outros tipos (nível 1)

$(S = S + m;)$

DMEM m – Libera memória alocada para variáveis na pilha

$(S = S - m;)$

Uso da MP

Comandos da MP para suporte ao Endereçamento Relativo

LOAD m, n – Deve informar qual o registrador de base m e o deslocamento n a ser usado para acessar o conteúdo da variável

$$(S = S + 1; M[S] = M[D[m] + n];)$$

STOR m, n – Deve informar qual o registrador de base m e o deslocamento n a ser usado para armazenar o valor na variável

$$(M[D[m] + n] = M[S]; S = S - 1;)$$

Uso da MP

- ❑ Comandos da MP para suporte ao Endereçamento Relativo
 - Chamada e retorno de blocos

CALL *m* – Como antes.

$(S = S+1; M[S]=I+1; I=L)$

INIPR *k* – Início de procedimento. Será sempre a primeira instrução de um procedimento. O parâmetro *k* indica o nível (escopo) do procedimento.

$(S = S+1; M[S]=D[k]; D[k]=S + 1;)$

RET *k, n* – Restaura o conteúdo do registrador de base e retorna do procedimento liberando o espaço usado na pilha para os parâmetros

$(D[k] = M[S]; I = M[S] - 1; S = S - (n + 2);)$

Uso da MP

❑ Comandos da MP para suporte ao Endereçamento Relativo

■ Início de Programa

INIP – Passa a iniciar também o registrador de base do nível 0.

(**S** = -1; **D**[0] = 0;)

■ Instruções para passagem e acesso a variáveis por referência

LDA **m**, **n** – Carrega endereço

(**S** = **S** + 1; **M**[**S**] = **D**[**m**] + **n**;)

LOADI **m**, **n** – Carrega valor indiretamente

(**S** = **S** + 1; **M**[**S**] = **M**[**M**[**D**[**m**] + **n**];)

STORI **m**, **n** – Armazena indiretamente

(**M**[**M**[**D**[**m**] + **n**] = **M**[**S**]; **S** = **S** - 1;)

Uso da MP

- Exemplo de programa em *C-like*

```
int x, y;

void P() {
    int z;

    z=x; x=x-1;
    if (z>1) P();
    else y=1;
    y=y*z;
}

void main() {
    read(x);
    P();
    write(x, y);
}
```

Código da MP?

Uso da MP

INIP		
AMEM	2	<i>int x,y</i>
GOTO	L1	
LABEL	L2	<i>void p()</i>
INIPR	1	
AMEM	1	<i>int z</i>
LOAD	0,0	
STOR	1,0	<i>z=x</i>
LOAD	0,0	
PUSH	1	
SUB		
STOR	0,0	<i>x=x-1</i>
LOAD	1,0	
PUSH	1	
SUB		
COPY		
GOTRUE	L3	
POP		
PUSH	0	
LABEL	L3	
GOFALSE	L4	<i>if (z>1)</i>
CALL	L2	<i>p()</i>
GOTO	L5	
LABEL	L4	<i>else</i>
PUSH	1	
STOR	0,1	<i>y:=1</i>

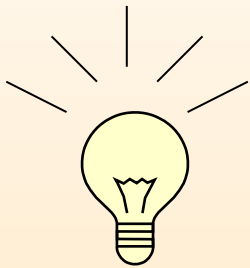
LABEL	L5	
LOAD	0,1	
LOAD	1,0	
MUL		
STOR	0,1	<i>y:=y*z</i>
DMEM	1	<i>end</i>
RET	1,0	
LABEL	L1	
GOTO	L6	
LABEL	L7	<i>void main()</i>
INIPR	1	
GET		
STOR	0,0	<i>read(x)</i>
CALL	L2	<i>p()</i>
LOAD	0,0	
PUT		
LOAD	0,1	
PUT		<i>write(x,y)</i>
RET	1,0	
LABEL	L6	
CALL	L7	
DMEM	2	
HALT		

Uso da MP

- Suporte da MP a blocos com parâmetros
 - Em muitas linguagens imperativas parâmetros formais passados por valor comportam-se como variáveis locais
 - Essas variáveis são preenchidas com os valores dos parâmetros efetivos
 - Na chamada do bloco, os parâmetros efetivos são expressões e é natural que seus valores sejam avaliados e deixados na pilha

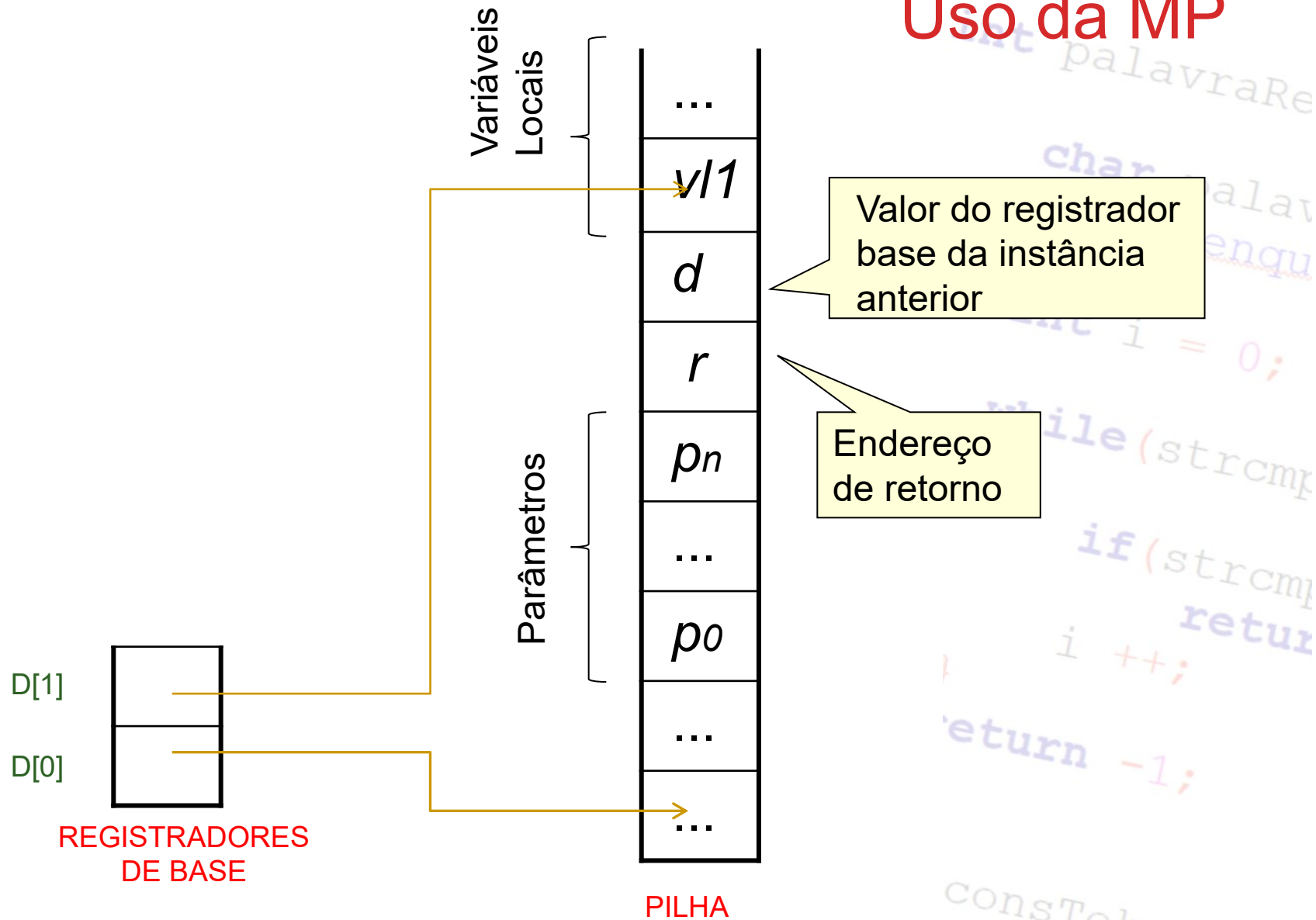
Uso da MP

❑ Solução:



Usar a pilha para alocar espaço para os parâmetros passados por valor e usar deslocamentos negativos em relação ao conteúdo do registrador base do procedimento para acessar os parâmetros: -3, -4, -5. **Por que iniciar em -3?!**

Uso da MP



Exercício

- Forneça a tradução para o trecho de programa *C-like* a seguir para código da MP, como se fosse produzida por um esquema de tradução (código gerado por tradutor):

```
int i;  
  
int fat(int n) {  
    int z;  
  
    if (z<=1)  
        z=1;  
    else  
        z=n*fat(n-1);  
    return z;  
}  
  
void main() {  
    int x, y;  
    x = 5;  
    y = fat(x);  
}
```

- Suporte da MP à indexação de vetores e matrizes
 - Como a operação de indexação envolve operações aritméticas para cálculo de endereço em tempo de execução, três instruções de acesso à memória com operando na pilha são necessárias

LDDLC *n* – Carrega o deslocamento na pilha*

($S = S + 1$; $M[S] = n$;))

LDSTK *m* – Carrega valor cujo deslocamento está no topo da pilha – que será sobreposto

($M[S] = M[M[D[m]] + M[S]]$;))

STSTK *m* – Armazena valor do topo da pilha na posição cujo deslocamento está no subtopo da pilha

($M[M[D[m]] + M[S-1]] = M[S]$; $S = S - 2$;))

(*) Mesma semântica do PUSH

Uso da MP

- Suporte da MP à indexação de vetores e matrizes
 - Exemplo – indexação com constante

```
...  
void P() {  
    int a[100];  
    int x;  
    ...  
    a[5] = 3; (1)  
    ...  
    x = a[5]; (2)  
}
```

```
(1)  
...  
LDDLC 0  
PUSH 5  
ADD  
PUSH 3  
STSTK 1  
...
```

```
(2)  
...  
LDDLC 0  
PUSH 5  
ADD  
LDSTK 1  
STOR 1, 100  
...
```

Uso da MP

- Suporte da MP à indexação de vetores e matrizes
 - Exemplo – indexação com variável

```
...  
void P() {  
    int a[100];  
    int x, i;  
    ...  
    a[i] = 0; (1)  
    ...  
    x = a[i]; (2)  
}
```

```
(1)  
...  
LDDLC 0  
LOAD 1, 101  
ADD  
PUSH 0  
STSTK 1  
...
```

```
(2)  
...  
LDDLC 0  
LOAD 1, 101  
ADD  
LDSTK 1  
STOR 1, 100  
...
```